DSA06 - Data Handling and Visualization

Practice Class Activity 2: Relationship Workshop in R

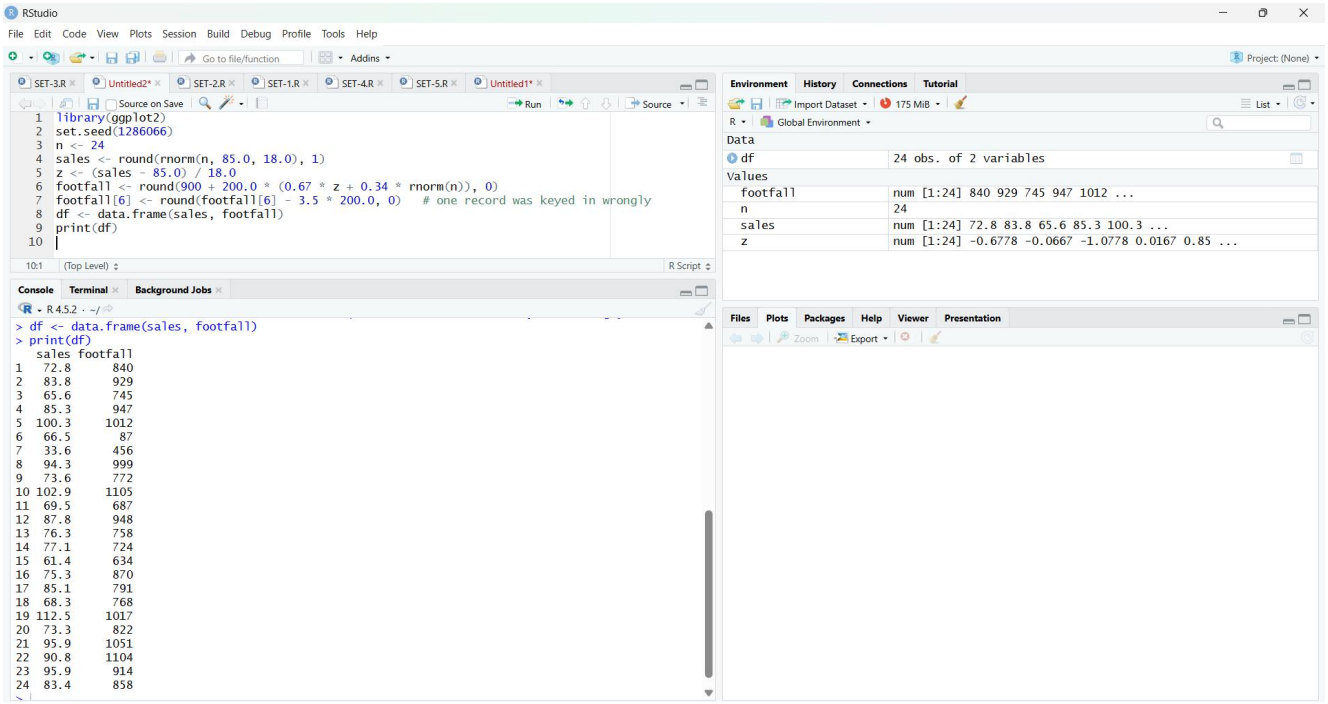
Department	AIDS
Course Code / Course Name	DSA0602 / Data Handling and Visualization
Name	B KIRTHANAA
Register Number	192324244
Roll Number	06
Slot	D
Tool	R (ggplot2, igraph, Rtsne, base R)

Q1 Scatterplot

Scenario: study weekly sales (Rs lakh) against footfall (visitors per week) across 24 weekly records of a retail store.

```
library(ggplot2)
set.seed(1286066)
n <- 24
sales <- round(rnorm(n, 85.0, 18.0), 1)
z <- (sales - 85.0) / 18.0
footfall <- round(900 + 200.0 * (0.67 * z + 0.34 * rnorm(n)), 0)
footfall[6] <- round(footfall[6] - 3.5 * 200.0, 0)   # one record was keyed in wrongly
df <- data.frame(sales, footfall)
print(df)
```

OUTPUT:

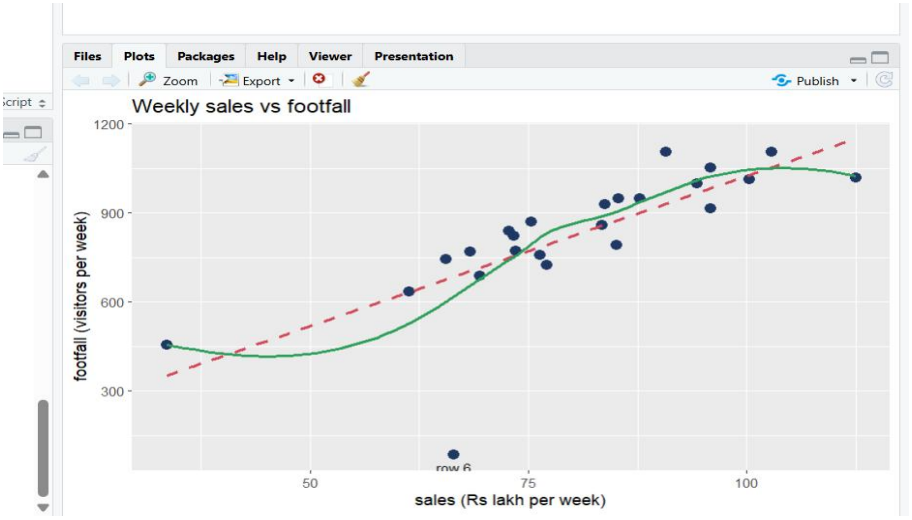


Task 1 [1.5]: Draw a scatterplot of sales (x) against footfall (y) with a straight-line fit (geom_smooth(method = "lm")) and a loess curve; label axes with units.

```
ggplot(df, aes(sales, footfall)) +
  geom_point(size = 3, colour = "#1F3864") +
  geom_text(data = df[6, ], aes(label = "row 6"), vjust = 1.8, size = 3.2) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE,
              colour = "#D1495B", linetype = "dashed") +
  geom_smooth(method = "loess", formula = y ~ x, se = FALSE,
              colour = "#2E9E5B") +
  labs(title = "Weekly sales vs footfall",
```

```
       x = "sales (Rs lakh per week)", y = "footfall (visitors per week)")
```

OUTPUT:



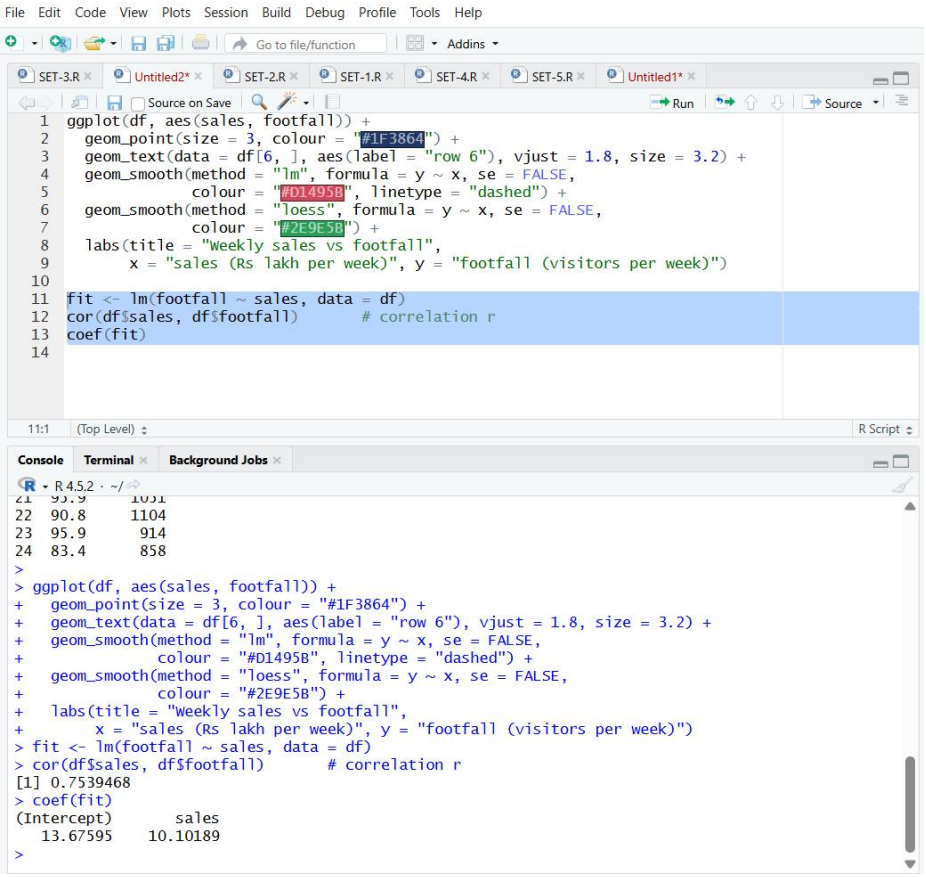
Output 1: sales vs footfall with straight-line (dashed) and loess (solid) fits

Ans: The points rise from lower left to upper right. The dashed red line is the straight-line fit and the green line is the loess curve. One point (row 6) sits far below both lines.

Task 2 [1]: Report the correlation r and the slope of the line, and describe the strength and direction in one sentence.

```
fit <- lm(footfall ~ sales, data = df)
cor(df$sales, df$footfall)       # correlation r
coef(fit)                        # intercept and slope
```

OUTPUT:

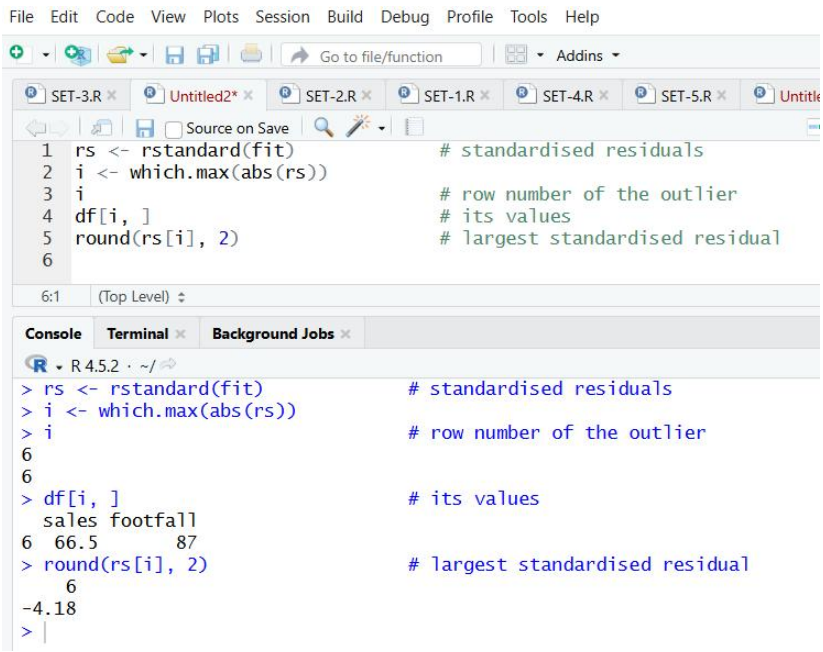


Ans: r = 0.75 and the slope = 10.10. There is a strong positive straight-line link: on average each extra Rs 1 lakh of weekly sales goes with about 10 more visitors. The wrong record weakens this: without row 6, r rises to 0.91 and the slope is 8.66.

Task 3 [1]: Flag the outlier: give its row number and values, and the largest standardised residual (rstandard) from the straight-line fit.

```
rs <- rstandard(fit)              # standardised residuals
i <- which.max(abs(rs))
i                                 # row number of the outlier
df[i, ]                           # its values
round(rs[i], 2)                   # largest standardised residual
```

OUTPUT:

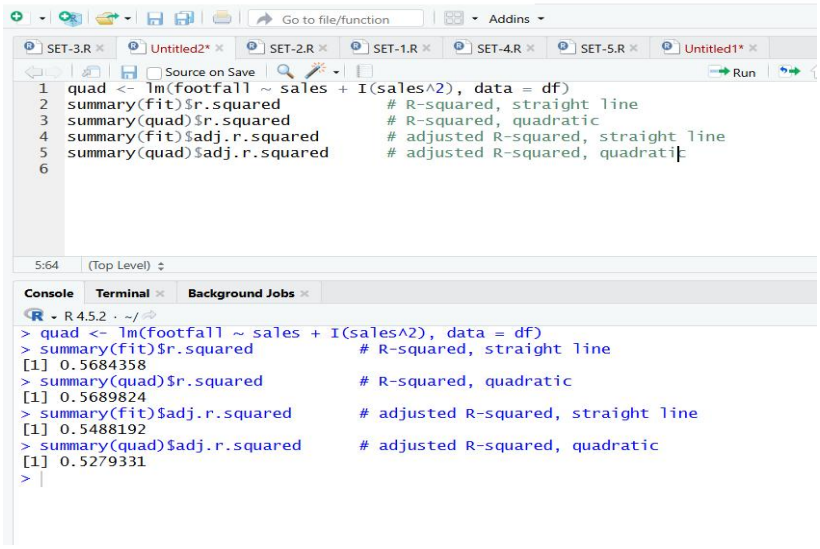


Ans: The outlier is row 6 (sales = 66.5, footfall = 87). Its standardised residual is -4.18, the largest in the data and far beyond the usual +/-2 limit. The line predicts about 685 visitors for sales of 66.5, so a footfall of 87 was keyed in wrongly.

Task 4 [1]: Fit a quadratic model (y ~ x + I(x^2)) and compare R-squared with the straight line; state whether a linear or non-linear trend fits better.

```
quad <- lm(footfall ~ sales + I(sales^2), data = df)
summary(fit)$r.squared            # R-squared, straight line
summary(quad)$r.squared           # R-squared, quadratic
summary(fit)$adj.r.squared        # adjusted R-squared, straight line
summary(quad)$adj.r.squared       # adjusted R-squared, quadratic
```

OUTPUT:



Ans: R-squared is 0.568 for the straight line and 0.569 for the quadratic, a gain of only 0.0005. The adjusted R-squared actually falls (0.549 to 0.528). A linear trend fits better: the bend in the loess curve is caused by the one wrong record, not by a real curve.

Task 5 [0.5]: Give one recommendation for the Sales Director that follows from the chart.

Ans: Recommendation: correct or re-check the footfall figure for row 6 before reporting, then use the straight-line relationship (about 9 to 10 extra visitors per Rs 1 lakh of sales) to plan staffing and promotions, and add a simple data-entry check for impossible footfall values.

Q2 Correlogram (correlation matrix heatmap)

Scenario: study six measures for 36 circle-periods (here n = 30 records) at a telecom firm: tower_density, drop_rate, traffic_tb, users_lakh, churn_pct and arpu.

Case data (code as given, first 10 rows printed):

```
library(ggplot2)
set.seed(1286144)
n <- 30
f1 <- rnorm(n)
f2 <- rnorm(n)
tower_density <- round(14.0 + 4.0 * (0.77 * f1 + 0.64 * rnorm(n)), 0)
drop_rate <- round(2.6 + 0.7 * (0.82 * f2 + 0.58 * rnorm(n)), 1)
traffic_tb <- round(320 + 70.0 * (-0.77 * f1 + 0.64 * rnorm(n)), 0)
users_lakh <- round(24.0 + 6.0 * (0.77 * f1 + 0.63 * rnorm(n)), 1)
churn_pct <- pmax(round(1.8 + 0.6 * (0.82 * f2 + 0.58 * rnorm(n)), 1), 0.1)
arpu <- round(185 + 25.0 * (0.76 * f2 + 0.65 * rnorm(n)), 0)
df <- data.frame(tower_density, drop_rate, traffic_tb, users_lakh, churn_pct, arpu)
print(head(df, 10))
```

OUTPUT:

Console | Terminal | Background Jobs

R 4.5.2 · ~/

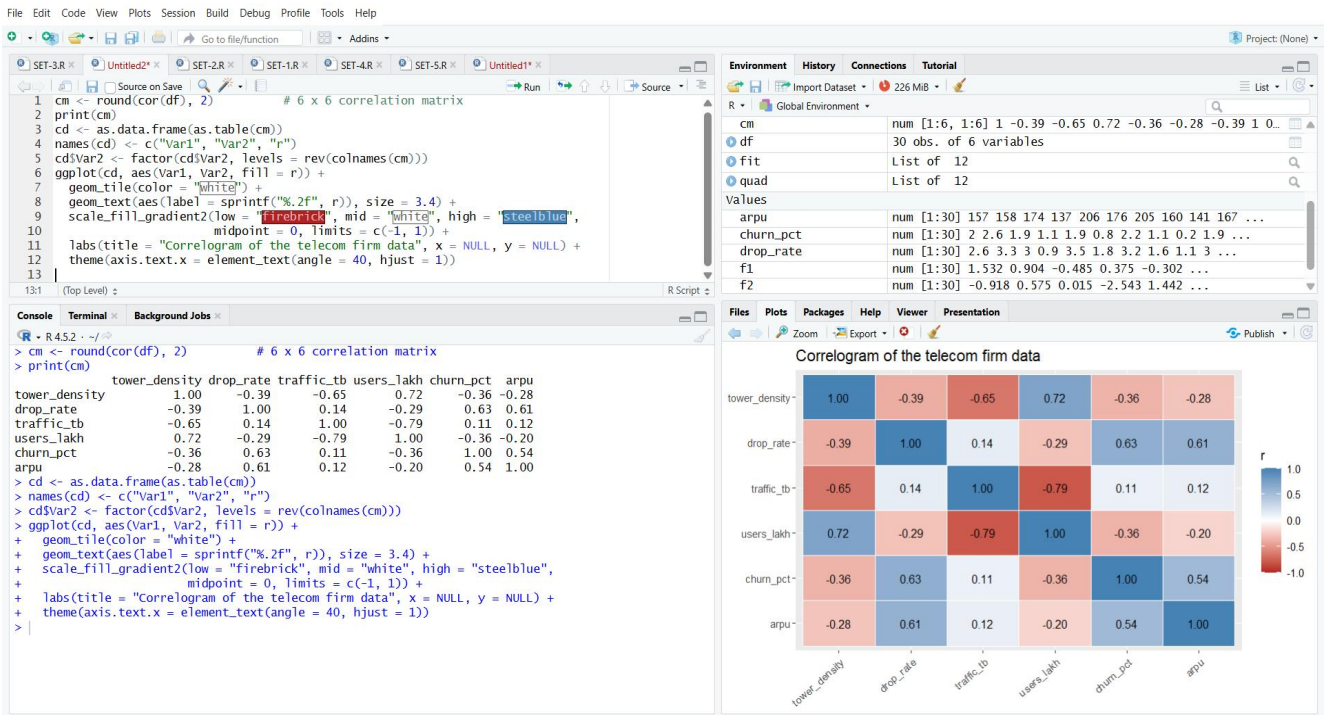
```
> library(ggplot2)
> set.seed(1286144)
> n <- 30
> f1 <- rnorm(n)
> f2 <- rnorm(n)
> tower_density <- round(14.0 + 4.0 * (0.77 * f1 + 0.64 * rnorm(n)), 0)
> drop_rate <- round(2.6 + 0.7 * (0.82 * f2 + 0.58 * rnorm(n)), 1)
> traffic_tb <- round(320 + 70.0 * (-0.77 * f1 + 0.64 * rnorm(n)), 0)
> users_lakh <- round(24.0 + 6.0 * (0.77 * f1 + 0.63 * rnorm(n)), 1)
> churn_pct <- pmax(round(1.8 + 0.6 * (0.82 * f2 + 0.58 * rnorm(n)), 1), 0.1)
> arpu <- round(185 + 25.0 * (0.76 * f2 + 0.65 * rnorm(n)), 0)
> df <- data.frame(tower_density, drop_rate, traffic_tb, users_lakh, churn_pct, arpu)
> print(head(df, 10))
   tower_density drop_rate traffic_tb users_lakh churn_pct arpu
1             19       2.6        166       28.6       2.0  157
2             14       3.3        272       24.8       2.6  158
3             18       3.0        259       22.3       1.9  174
4             16       0.9        319       24.0       1.1  137
5             14       3.5        332       22.7       1.9  206
6             19       1.8        233       25.2       0.8  176
7             18       3.2        205       25.4       2.2  205
8             12       1.6        348       23.4       1.1  160
9             17       1.1        333       28.5       0.2  141
10             5       3.0        337       18.2       1.9  167
> |
```

Task 1 [1.5]: Compute cor(df), round to 2 decimals, and draw a correlogram (ggplot2 geom_tile with geom_text, or corrplot) with a diverging colour scale centred at 0.

```
cm <- round(cor(df), 2)          # 6 x 6 correlation matrix
print(cm)
cd <- as.data.frame(as.table(cm))
names(cd) <- c("Var1", "Var2", "r")
cd$Var2 <- factor(cd$Var2, levels = rev(colnames(cm)))
ggplot(cd, aes(Var1, Var2, fill = r)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 3.4) +
  scale_fill_gradient2(low = "firebrick", mid = "white", high = "steelblue",
                       midpoint = 0, limits = c(-1, 1)) +
  labs(title = "Correlogram of the telecom firm data", x = NULL, y = NULL) +
```

```
  theme(axis.text.x = element_text(angle = 40, hjust = 1))
```

OUTPUT:



Ans: The 6 x 6 matrix is printed above and shown as the correlogram. Blue cells are positive, red cells are negative and white cells are near zero. The darkest cells are traffic_tb with users_lakh (red) and tower_density with users_lakh (blue).

Task 2 [1.5]: Identify the two strongest correlated variable pairs (by absolute r) and report their r.

```
ut <- which(upper.tri(cm), arr.ind = TRUE)      # off-diagonal pairs only
pairs_df <- data.frame(var1 = rownames(cm)[ut[, 1]],
                       var2 = colnames(cm)[ut[, 2]],
                       r = cm[ut])
pairs_df$abs_r <- abs(pairs_df$r)
head(pairs_df[order(-pairs_df$abs_r), c("var1", "var2", "r")], 2)
```

OUTPUT:

```
           var1       var2     r
6    traffic_tb users_lakh -0.79
4 tower_density users_lakh  0.72
```

Ans: The strongest pair is traffic_tb and users_lakh (r = -0.79, strong negative). The second is tower_density and users_lakh (r = 0.72, strong positive).

Task 3 [1]: Identify one surprising weak or negative correlation, report its r and explain in one sentence what it implies.

```
pairs_df[order(pairs_df$abs_r), c("var1", "var2", "r")][1:4, ]
```

OUTPUT:

```
         var1       var2     r
9  traffic_tb  churn_pct  0.11
13 traffic_tb       arpu  0.12
3   drop_rate traffic_tb  0.14
14 users_lakh       arpu -0.20
```

Ans: traffic_tb and drop_rate have r = 0.14 (weak). Heavier traffic would be expected to cause more dropped calls, but the two hardly move together, so network load alone does not explain the drop rate. (The pair traffic_tb and churn_pct is also weak, r = 0.11.)

Task 4 [1]: Give one recommendation for the Network Planner based on the matrix.

Ans: Recommendation: drop_rate moves with churn_pct (r = 0.63) and arpu (r = 0.61), so treat call drops as the priority to fix, rather than traffic load, which is almost unrelated to them (r = 0.14). Also note that tower_density goes with users_lakh (r = 0.72); a correlation is not proof of cause, so test any tower upgrade on a pilot circle first.

Q3 Dimension Reduction

Scenario: six measures for 36 hotels in three known groups (Group A, B, C), 12 hotels each.

Case data (code as given, first 8 rows printed):

```
library(ggplot2)
set.seed(1286159)
n <- 36
grp <- rep(c("Group A", "Group B", "Group C"), each = 12)
g <- rep(c(-1, 0, 1), each = 12)
footfall_k <- round(120.0 + 38.0 * (rnorm(n) + 1.00 * g), 1)
spend_rs <- round(3200 + 600.0 * (rnorm(n) + 0.40 * g), 0)
stay_nights <- round(2.8 + 0.7 * (rnorm(n) + 0.11 * g), 1)
rating10 <- round(7.6 + 0.8 * (rnorm(n) + 1.10 * g), 1)
occupancy_pct <- round(68.0 + 10.0 * (rnorm(n) + 1.00 * g), 1)
complaints <- round(8.0 + 1.2 * (rnorm(n) + 0.07 * g), 0)
df <- data.frame(footfall_k, spend_rs, stay_nights, rating10, occupancy_pct, complaints)
print(head(df, 8))
```

OUTPUT:

```
  footfall_k spend_rs stay_nights rating10 occupancy_pct complaints
1       78.3     2994         2.9      6.9          46.7          9
2       40.3     3185         3.1      7.1          84.5          7
3      129.1     2616         3.0      6.9          70.2          8
4       59.9     2462         1.2      6.6          57.3          7
5       38.1     3477         1.6      6.8          52.7          9
6       55.8     3224         3.8      7.0          48.8          7
7       80.2     3446         2.4      6.4          63.7          9
8      119.6     1776         3.1      5.9          78.8         10
```

Task 1 [1]: Run PCA on the scaled data (prcomp(df, scale. = TRUE)). Report the percentage of variance for PC1 and PC2 and the number of components needed to reach at least 80%.

```
pca <- prcomp(df, scale. = TRUE)
imp <- summary(pca)$importance
round(imp, 3)                                   # SD, proportion, cumulative
round(100 * imp[2, 1:2], 1)                     # % variance of PC1 and PC2
which(imp[3, ] >= 0.80)[1]                      # components needed for 80%
```

OUTPUT:

```
                          PC1   PC2   PC3   PC4   PC5   PC6
Standard deviation     1.476 1.170 0.958 0.820 0.730 0.575
Proportion of Variance 0.363 0.228 0.153 0.112 0.089 0.055
Cumulative Proportion  0.363 0.591 0.744 0.856 0.945 1.000
 PC1  PC2
36.3 22.8
PC4
  4
```

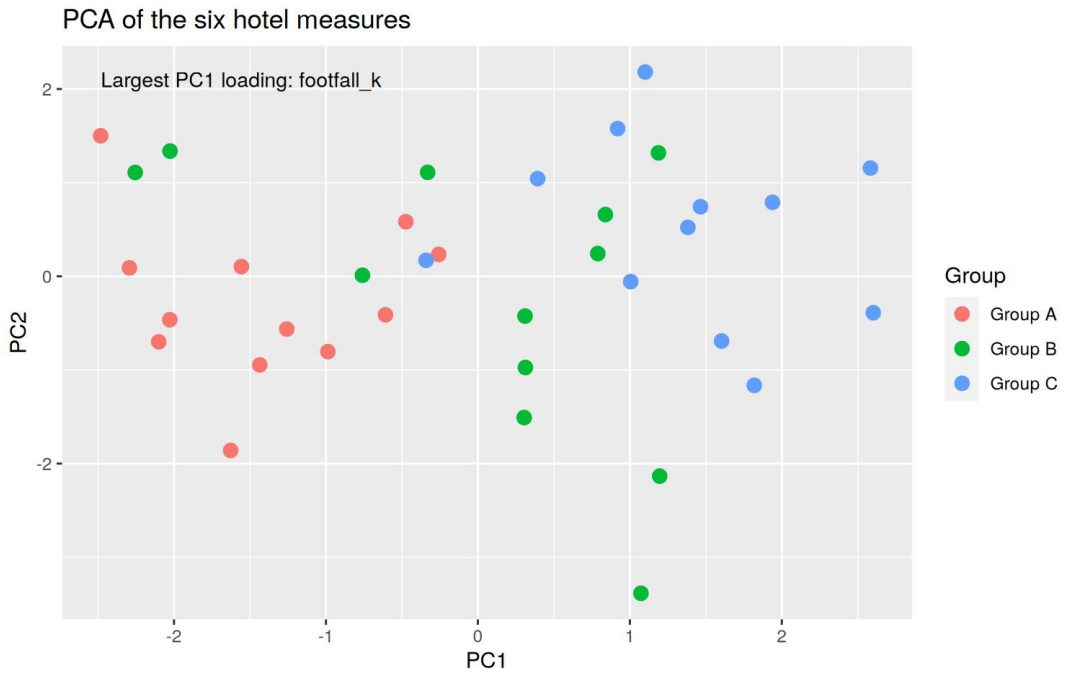
Ans: PC1 explains 36.3% and PC2 explains 22.8% of the variance (59.1% together). PC3 brings the total to 74.4%, so 4 components are needed to pass 80% (cumulative 85.6%).

Task 2 [1]: Plot PC1 against PC2 coloured by grp, then label the variable with the largest absolute loading on PC1.

```
sc <- data.frame(pca$x[, 1:2], grp = grp)
load1 <- pca$rotation[, 1]
round(load1, 2)                                 # PC1 loadings
top <- names(which.max(abs(load1)))
top                                             # largest absolute loading on PC1
ggplot(sc, aes(PC1, PC2, colour = grp)) +
  geom_point(size = 3) +
  annotate("text", x = min(sc$PC1), y = max(sc$PC2), hjust = 0, vjust = 1,
           label = paste("Largest PC1 loading:", top), size = 3.6) +
  labs(title = "PCA of the six hotel measures",
```

```
      x = "PC1", y = "PC2", colour = "Group")
```

OUTPUT:



Output 3: PCA scores, PC1 vs PC2, coloured by group

```
   footfall_k      spend_rs   stay_nights      rating10 occupancy_pct
         0.59          0.33          0.30          0.43          0.51
   complaints
         0.09
[1] "footfall_k"
```

Ans: The variable with the largest absolute loading on PC1 is footfall_k (0.59), followed by occupancy_pct (0.51) and rating10 (0.43). It is labelled on the plot. Group A lies on the left, Group C on the right and Group B in the middle.

Task 3 [1]: Judge how well PCA separates the groups: use the share of variance in each of PC1 and PC2 explained by grp (R-squared from lm(score ~ grp)).

```
r2_pc1 <- summary(lm(PC1 ~ grp, data = sc))$r.squared
r2_pc2 <- summary(lm(PC2 ~ grp, data = sc))$r.squared
round(c(PC1 = r2_pc1, PC2 = r2_pc2), 3)
```

OUTPUT:

```
  PC1   PC2
0.617 0.091
```

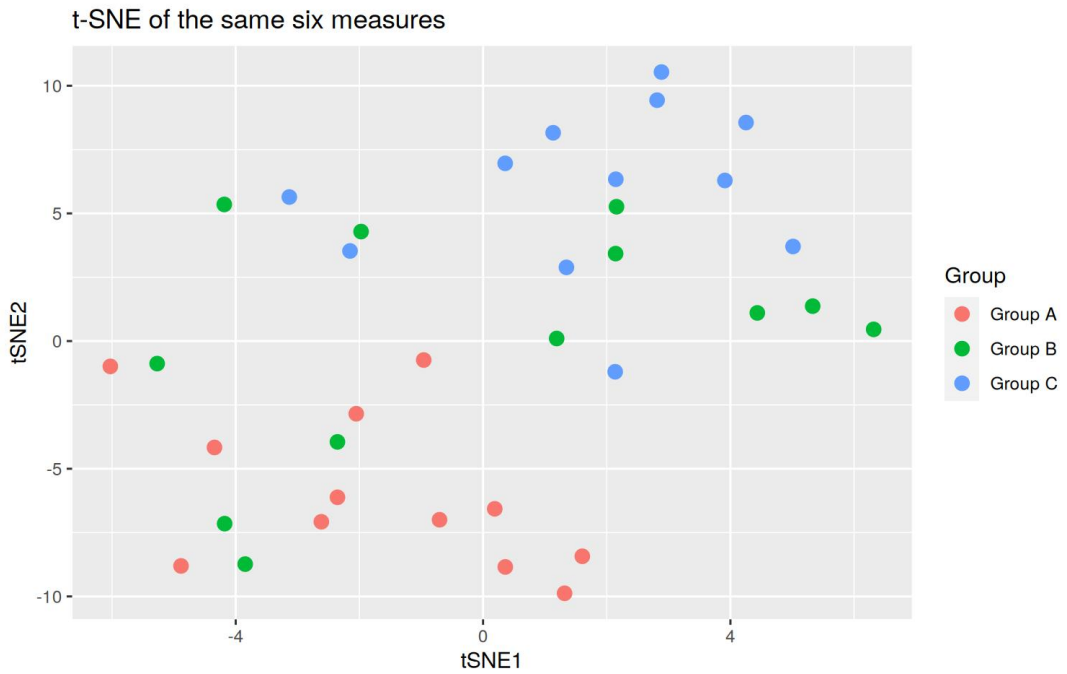
Ans: grp explains 61.7% of the variance of PC1 but only 9.1% of PC2. PCA separates the groups well along PC1 (A to B to C in order), but PC2 carries almost no group information and Group B still overlaps both neighbours.

Task 4 [1.5]: Make a t-SNE (Rtsne) or UMAP plot of the same data and explain the variance-versus-neighbourhood trade-off between it and PCA; give one recommendation for the analyst.

```
library(Rtsne)
set.seed(42)
tsne <- Rtsne(scale(df), perplexity = 8, check_duplicates = FALSE)
ts <- data.frame(tSNE1 = tsne$Y[, 1], tSNE2 = tsne$Y[, 2], grp = grp)
ggplot(ts, aes(tSNE1, tSNE2, colour = grp)) +
  geom_point(size = 3) +
  labs(title = "t-SNE of the same six measures", colour = "Group")
round(summary(lm(tSNE1 ~ grp, data = ts))$r.squared, 3)   # group share on tSNE1
```

```
round(summary(lm(tSNE2 ~ grp, data = ts))$r.squared, 3)   # group share on tSNE2
```

OUTPUT:



Output 4: t-SNE of the same data (perplexity = 8)

```
[1] 0.182
[1] 0.649
```

Ans: PCA is linear and keeps as much global variance as possible, so its axes can be read through the loadings, but a 2-D view keeps only 59% of the variance. t-SNE keeps each point's local neighbourhood and can bend the data to show clusters; here grp explains 18.2% of tSNE1 and 64.9% of tSNE2. The price is that distances between clusters, cluster sizes and the axes have no direct meaning, and the picture changes with perplexity and seed. Recommendation: use PCA (PC1, driven by footfall, occupancy and rating) as the hotel performance score, and use t-SNE only as a visual check for groups, not for reporting numbers.

Q4 Network Graph

Scenario: parcels delivered between 7 hubs; each row of the edge list is a link with a weight (parcels delivered, in thousands).

Case data (code as given, edge list printed):

```
library(igraph)
set.seed(1286229)
nodes <- c("Mumbai", "Ahmedabad", "Chennai", "Pune", "Kolkata", "Delhi", "Bengaluru")
pairs <- t(combn(nodes, 2))
u <- runif(nrow(pairs))
keep <- (u < 0.25 | ((pairs[, 1] == "Kolkata" | pairs[, 2] == "Kolkata") & u < 0.92))
edges <- data.frame(from = pairs[keep, 1], to = pairs[keep, 2],
                    weight = round(runif(sum(keep), 1, 20)))
print(edges)
```

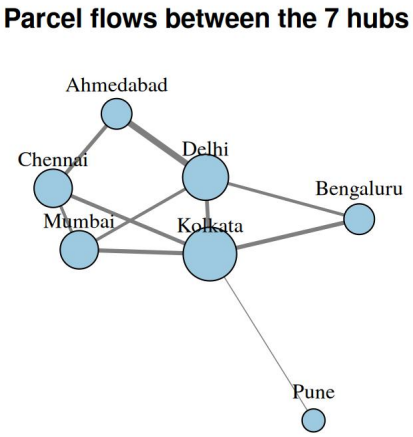
OUTPUT:

```
        from        to weight
1     Mumbai   Chennai      9
2     Mumbai   Kolkata     12
3     Mumbai     Delhi      9
4  Ahmedabad   Chennai     11
5  Ahmedabad     Delhi     18
6    Chennai   Kolkata     11
7       Pune   Kolkata      3
8    Kolkata     Delhi     11
9    Kolkata Bengaluru     12
10     Delhi Bengaluru      9
```

Task 1 [1.5]: Build the network with igraph (graph_from_data_frame). Draw it with node size proportional to degree and edge width proportional to weight, with node labels.

```
g <- graph_from_data_frame(edges, directed = FALSE,
                           vertices = data.frame(name = nodes))
deg <- degree(g)
set.seed(7)
plot(g, layout = layout_with_fr(g),
     vertex.size = 10 + 5 * deg, vertex.color = "#9ECAE1",
     vertex.label.color = "black", vertex.label.cex = 0.95,
     vertex.label.dist = 2.6, vertex.label.degree = -pi/2,
     edge.width = E(g)$weight / 4, edge.color = "grey50",
     main = "Parcel flows between the 7 hubs")
```

OUTPUT:



Output 5: Parcel network (node size = degree, edge width = weight)

Ans: The network is drawn with larger circles for hubs with more links and thicker lines for heavier flows. Kolkata is the largest node and sits at the centre; Pune hangs off the network by a single thin link.

Task 2 [1.5]: Compute degree and betweenness for every node and name the most central node, justifying it with both numbers.

```
cen <- data.frame(degree = degree(g),
                  betweenness = round(betweenness(g, weights = NA), 2))
cen[order(-cen$betweenness), ]
rownames(cen)[which.max(cen$betweenness)]        # most central node
```

OUTPUT:

```
          degree betweenness
Kolkata        5        6.83
Delhi          4        3.00
Chennai        3        1.50
Mumbai         3        0.33
Ahmedabad      2        0.33
Pune           1        0.00
Bengaluru      2        0.00
[1] "Kolkata"
```

Ans: Kolkata is the most central node: it has the highest degree (5 links) and the highest betweenness (6.83), well above Delhi (degree 4, betweenness 3.00). Many shortest routes between other hubs pass through Kolkata.

Task 3 [1]: Report the number of edges, the density (edge_density) and the number of connected components.

```
ecount(g)                                        # number of edges
round(edge_density(g), 3)                        # density
components(g)$no                                 # connected components
```

OUTPUT:

```
[1] 10
[1] 0.476
[1] 1
```

Ans: The network has 10 edges, a density of 0.476 (about 48% of the 21 possible links exist) and 1 connected component, so every hub can currently reach every other hub.

Task 4 [1]: Delete the most central node (delete_vertices) and report how many components remain; give one recommendation for the Operations Head.

```
top <- rownames(cen)[which.max(cen$betweenness)]
g2 <- delete_vertices(g, top)
components(g2)$no                                # components after deletion
components(g2)$csize                             # size of each component
```

OUTPUT:

```
[1] 2
[1] 5 1
```

Ans: After deleting Kolkata, 2 components remain (sizes 5 and 1): Pune is cut off completely because Kolkata is its only link. Recommendation: reduce the dependence on Kolkata by adding a backup route for Pune (for example Pune to Mumbai) and by keeping spare capacity at Kolkata, since its failure would isolate Pune and take away the busiest transfer point.

Q5 Paired / Slope Chart

Scenario: patient wait time (minutes; lower is better) recorded before and after a process change at 6 departments.

Case data (code as given, data frame printed):

```
library(ggplot2)
set.seed(1286364)
entity <- c("Pharmacy", "Neurology", "Emergency", "Orthopaedics", "Paediatrics", "Cardiology")
n <- 6
before <- round(rnorm(n, 42.0, 5.0), 1)
change <- round(c(-1, -1, -1, -1, 1, 1) * runif(n, 0.4, 1.4) * 6.0, 1)
change[4] <- 0.1                                 # one was almost unchanged
after <- before + change
df <- data.frame(entity, before, after)
print(df)
```

OUTPUT:

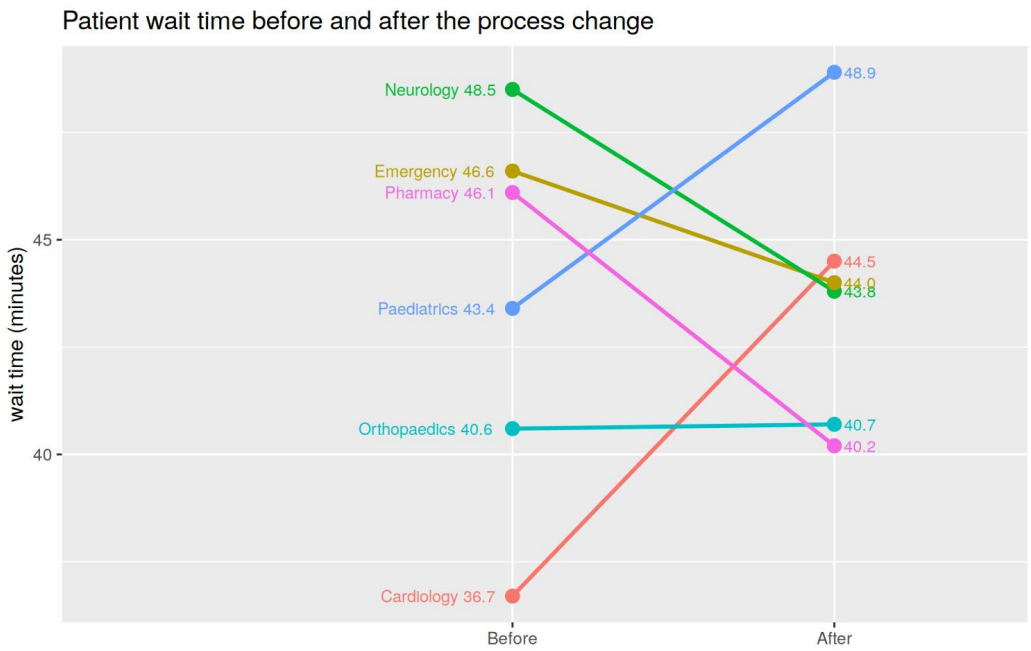
```
        entity before after
1     Pharmacy   46.1  40.2
2    Neurology   48.5  43.8
3    Emergency   46.6  44.0
4 Orthopaedics   40.6  40.7
5  Paediatrics   43.4  48.9
6   Cardiology   36.7  44.5
```

Task 1 [1.5]: Draw a paired slope chart (two points per entity joined by a line, geom_line(aes(group = entity))) with labelled endpoints.

```
long <- data.frame(entity = rep(df$entity, 2),
                   time = factor(rep(c("Before", "After"), each = n),
                                 levels = c("Before", "After")),
                   wait = c(df$before, df$after))
ggplot(long, aes(time, wait, group = entity, colour = entity)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  geom_text(data = subset(long, time == "Before"),
            aes(label = sprintf("%s %.1f", entity, wait)), hjust = 1.15, size = 3) +
  geom_text(data = subset(long, time == "After"),
            aes(label = sprintf("%.1f", wait)), hjust = -0.3, size = 3) +
  scale_x_discrete(expand = expansion(add = c(1.4, 0.6))) +
  labs(title = "Patient wait time before and after the process change",
       x = NULL, y = "wait time (minutes)") +
```

```
  theme(legend.position = "none")
```

OUTPUT:



Output 6: Slope chart of wait time before and after the change

Ans: Each line joins the before and after wait of one department. Lines that go down (Pharmacy, Neurology, Emergency) are improvements, the line that goes up steeply (Cardiology) is the worst change, and Orthopaedics is almost horizontal.

Task 2 [1]: Classify each entity as improved, worsened or flat (flat means the absolute change is below 1% of the 'before' value) and report the counts.

```
df$change <- round(df$after - df$before, 1)
df$flat_limit <- round(0.01 * df$before, 3)      # 1% of the 'before' value
df$status <- ifelse(abs(df$change) < df$flat_limit, "Flat",
                    ifelse(df$change < 0, "Improved", "Worsened"))
print(df)
table(df$status)
```

OUTPUT:

```
        entity before after change flat_limit   status
1     Pharmacy   46.1  40.2   -5.9      0.461 Improved
2    Neurology   48.5  43.8   -4.7      0.485 Improved
3    Emergency   46.6  44.0   -2.6      0.466 Improved
4 Orthopaedics   40.6  40.7    0.1      0.406     Flat
5  Paediatrics   43.4  48.9    5.5      0.434 Worsened
6   Cardiology   36.7  44.5    7.8      0.367 Worsened

    Flat Improved Worsened
       1        3        2
```

Ans: Improved = 3 (Pharmacy, Neurology, Emergency), worsened = 2 (Paediatrics, Cardiology) and flat = 1 (Orthopaedics, change +0.1 against a limit of 0.406).

Task 3 [1]: Quantify the largest change (entity, size, and percentage of its 'before' value).

```
k <- which.max(abs(df$change))
df$entity[k]                                  # entity with the largest change
df$change[k]                                  # size of the change (minutes)
round(100 * df$change[k] / df$before[k], 1)      # % of its 'before' value
```

OUTPUT:

```
[1] "Cardiology"
[1] 7.8
[1] 21.3
```

Ans: The largest change is Cardiology: its wait rose by 7.8 minutes, which is 21.3% of its before value (36.7 minutes). It is a worsening.

Task 4 [1.5]: Run a paired t-test (t.test(before, after, paired = TRUE)) and report t, p and the mean change; give one recommendation.

```
tt <- t.test(df$before, df$after, paired = TRUE)
tt
round(mean(df$after - df$before), 2)             # mean change (after - before)
```

OUTPUT:

```
      Paired t-test

data:  df$before and df$after
t = -0.014676, df = 5, p-value = 0.9889
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 -5.871693  5.805026
sample estimates:
mean difference
    -0.03333333

[1] 0.03
```

Ans: t = -0.015 (df = 5), p = 0.989, and the mean change (after minus before) is +0.03 minutes. So there is no overall change: the three improvements cancel the two worsenings. Recommendation: do not roll the process change out hospital-wide yet; find out what worked in Pharmacy, Neurology and Emergency and why Cardiology and Paediatrics got worse, then fix those two and re-test.

Q6 [Bubble Charts] (5 Marks) Bubble Chart

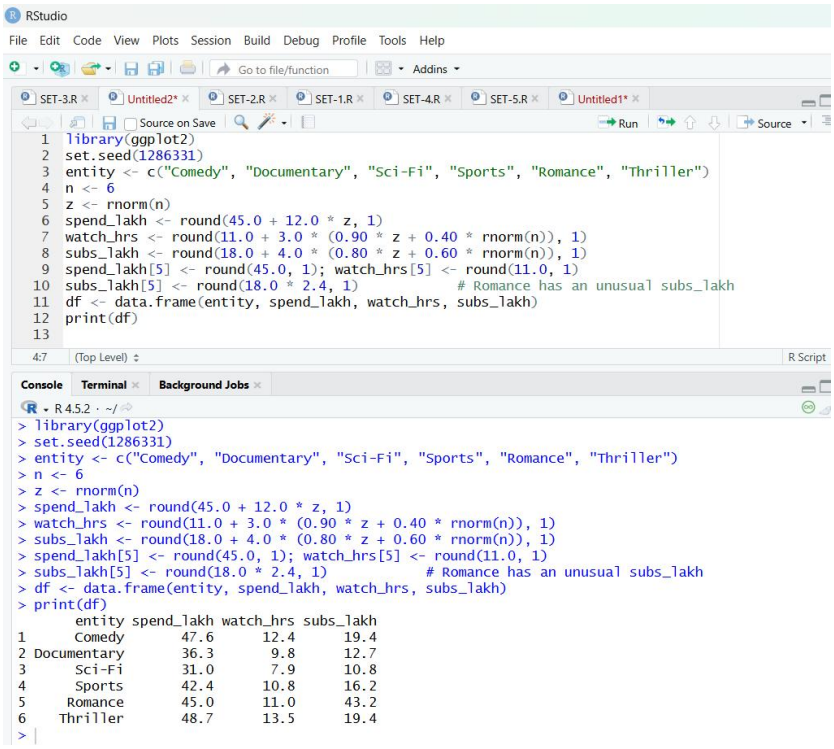
Scenario: compare 6 genres on marketing spend (Rs lakh, x), watch time (hours per user, y) and subscribers (lakh, bubble size).

Case data (code as given, data frame printed):

```
library(ggplot2)
set.seed(1286331)
entity <- c("Comedy", "Documentary", "Sci-Fi", "Sports", "Romance", "Thriller")
n <- 6
z <- rnorm(n)
spend_lakh <- round(45.0 + 12.0 * z, 1)
watch_hrs <- round(11.0 + 3.0 * (0.90 * z + 0.40 * rnorm(n)), 1)
subs_lakh <- round(18.0 + 4.0 * (0.80 * z + 0.60 * rnorm(n)), 1)
spend_lakh[5] <- round(45.0, 1); watch_hrs[5] <- round(11.0, 1)
subs_lakh[5] <- round(18.0 * 2.4, 1)            # Romance has an unusual subs_lakh
df <- data.frame(entity, spend_lakh, watch_hrs, subs_lakh)
```

```
print(df)
```

OUTPUT:



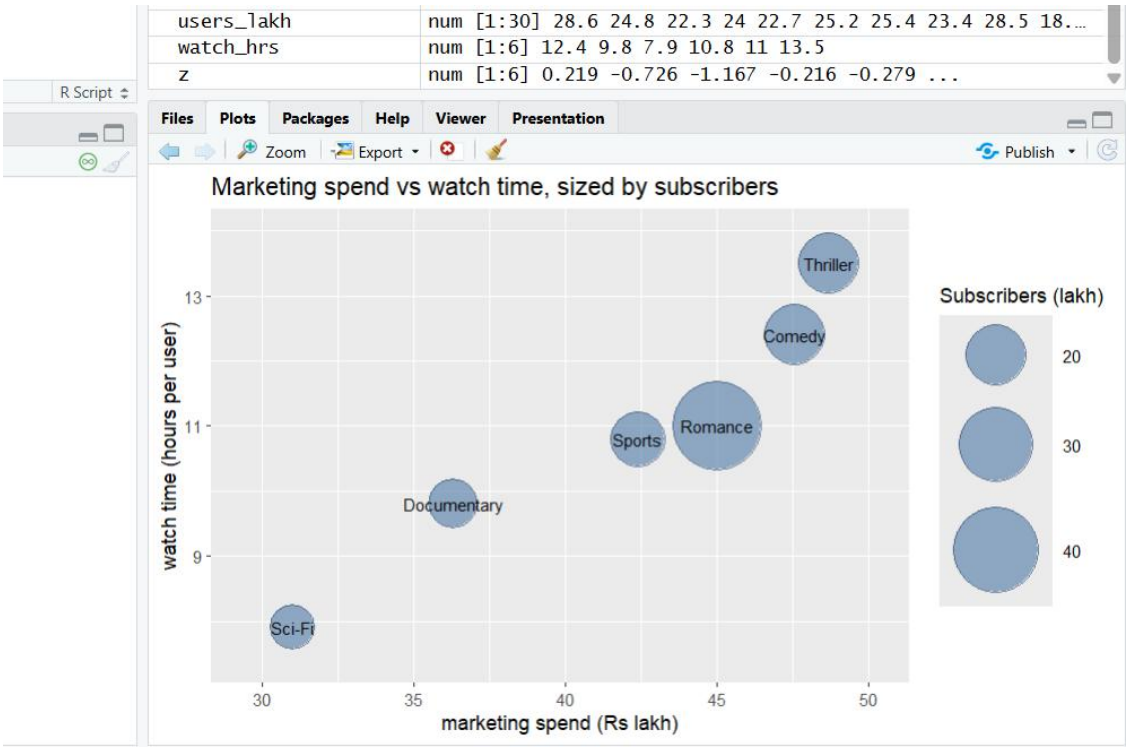
```
library(ggplot2)
set.seed(1286331)
entity <- c("Comedy", "Documentary", "Sci-Fi", "Sports", "Romance", "Thriller")
n <- 6
z <- rnorm(n)
spend_lakh <- round(45.0 + 12.0 * z, 1)
watch_hrs <- round(11.0 + 3.0 * (0.90 * z + 0.40 * rnorm(n)), 1)
subs_lakh <- round(18.0 + 4.0 * (0.80 * z + 0.60 * rnorm(n)), 1)
spend_lakh[5] <- round(45.0, 1); watch_hrs[5] <- round(11.0, 1)
subs_lakh[5] <- round(18.0 * 2.4, 1)          # Romance has an unusual subs_lakh
df <- data.frame(entity, spend_lakh, watch_hrs, subs_lakh)
print(df)
```

```
> library(ggplot2)
> set.seed(1286331)
> entity <- c("Comedy", "Documentary", "Sci-Fi", "Sports", "Romance", "Thriller")
> n <- 6
> z <- rnorm(n)
> spend_lakh <- round(45.0 + 12.0 * z, 1)
> watch_hrs <- round(11.0 + 3.0 * (0.90 * z + 0.40 * rnorm(n)), 1)
> subs_lakh <- round(18.0 + 4.0 * (0.80 * z + 0.60 * rnorm(n)), 1)
> spend_lakh[5] <- round(45.0, 1); watch_hrs[5] <- round(11.0, 1)
> subs_lakh[5] <- round(18.0 * 2.4, 1)          # Romance has an unusual subs_lakh
> df <- data.frame(entity, spend_lakh, watch_hrs, subs_lakh)
> print(df)
       entity spend_lakh watch_hrs subs_lakh
1      Comedy       47.6      12.4      19.4
2 Documentary       36.3       9.8      12.7
3      Sci-Fi       31.0       7.9      10.8
4      Sports       42.4      10.8      16.2
5     Romance       45.0      11.0      43.2
6    Thriller       48.7      13.5      19.4
>
```

Task 1 [1.5]: Draw a bubble chart with geom_point(aes(size = ...)) and scale_size_area(), labels on every bubble, and a legend for size.

```
ggplot(df, aes(spend_lakh, watch_hrs)) +
  geom_point(aes(size = subs_lakh), shape = 21, fill = "#4E79A7",
             colour = "#1F3864", alpha = 0.6) +
  geom_text(aes(label = entity), size = 3.2) +
  scale_size_area(max_size = 22, name = "Subscribers (lakh)") +
  scale_x_continuous(expand = expansion(mult = 0.15)) +
  scale_y_continuous(expand = expansion(mult = 0.15)) +
  labs(title = "Marketing spend vs watch time, sized by subscribers",
       x = "marketing spend (Rs lakh)", y = "watch time (hours per user)")
```

OUTPUT:



Output 7: Marketing spend vs watch time, bubble size = subscribers

Ans: Each genre is one bubble placed by spend and watch time, with area showing subscribers. The genres rise from Sci-Fi (bottom left) to Thriller (top right). Romance has by far the biggest bubble although it sits in the middle of the cloud.

Task 2 [0.5]: Report the correlation between the x and y variables.

```
round(cor(df$spend_lakh, df$watch_hrs), 2)
```

OUTPUT:

```
[1] 0.96
```

Ans: r = 0.96 between marketing spend and watch time: a very strong positive link (with only 6 genres, so treat it with care).

Task 3 [1.5]: Name the entity that is an outlier on the size variable even though its x/y position is near average, and show it with a z-score of size.

```
df$z_subs <- round(as.numeric(scale(df$subs_lakh)), 2)
df$z_spend <- round(as.numeric(scale(df$spend_lakh)), 2)
df$z_watch <- round(as.numeric(scale(df$watch_hrs)), 2)
df[, c("entity", "z_spend", "z_watch", "z_subs")]
df$entity[which.max(abs(df$z_subs))]             # size outlier
```

OUTPUT:

```
       entity z_spend z_watch z_subs
1      Comedy    0.83    0.76  -0.08
2 Documentary   -0.80   -0.56  -0.65
3      Sci-Fi   -1.57   -1.53  -0.81
4      Sports    0.08   -0.05  -0.35
5     Romance    0.46    0.05   1.95
6    Thriller    0.99    1.33  -0.08
[1] "Romance"
```

Ans: Romance. Its spend (z = 0.46) and watch time (z = 0.05) are close to average, but its subscribers have z = 1.95, the largest in the data and well apart from the other genres (all between -0.81 and -0.08).

Task 4 [1]: State how many times larger its size is than the median size, and explain why a plain scatterplot would have hidden it.

```
med <- median(df$subs_lakh)
med
df$subs_lakh[5]
round(df$subs_lakh[5] / med, 2)                  # times larger than the median
```

OUTPUT:

```
[1] 17.8
[1] 43.2
[1] 2.43
```

Ans: Romance has 43.2 lakh subscribers against a median of 17.8 lakh, which is 2.43 times larger. A plain scatterplot shows only x and y, and Romance sits on the middle of the trend (45.0, 11.0), so it would look like an ordinary genre; the size variable is what exposes it.

Task 5 [1]: Give one recommendation for the Content Head.

Ans: Recommendation: Romance has 2.4 times the median audience but only average spend and watch time, so its large subscriber base is under-used. Test more or better Romance titles and promotion to turn those subscribers into watch hours, and check the data in case the subscriber figure is a data error.

Q7 Hexbin Plot

Scenario: a large dataset of 3,000 loans: loan amount (Rs lakh, x) and applicant income (Rs thousand per month, y).

Case data (code as given, first 8 rows printed):

```
library(ggplot2)
set.seed(1286371)
n <- 3000
core <- (runif(n) < 0.85)
loan_lakh <- round(ifelse(core, rnorm(n, 38.0, 5.4), runif(n, 11.0, 65.0)), 1)
income_k <- round(ifelse(core, 52.0 - 0.75 * (loan_lakh - 38.0) / 5.4 +
                           sqrt(1 - 0.56) * 7.2 * rnorm(n), runif(n, 16.0, 88.0)), 1)
```

```
df <- data.frame(loan_lakh, income_k)
print(head(df, 8))
nrow(df)
```

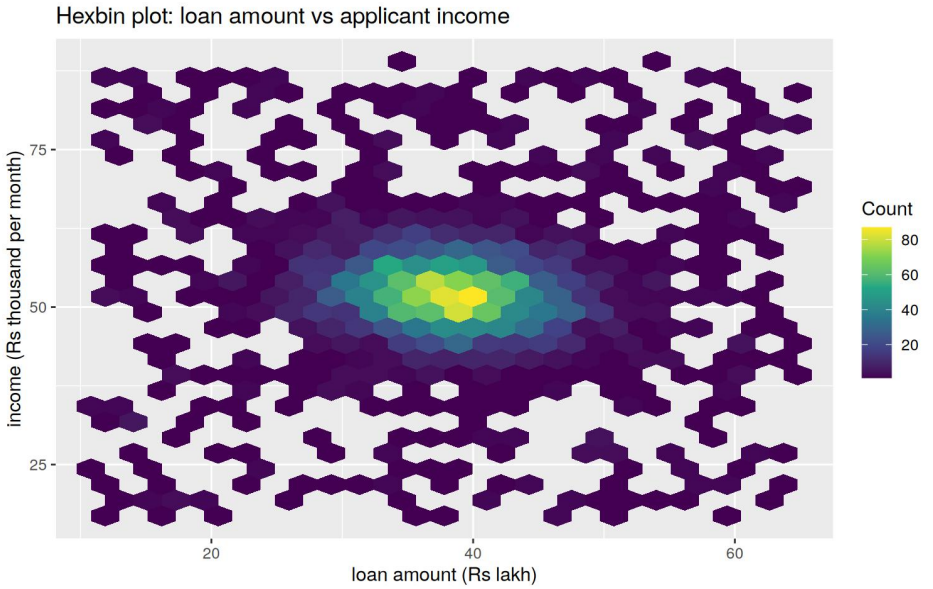
OUTPUT:

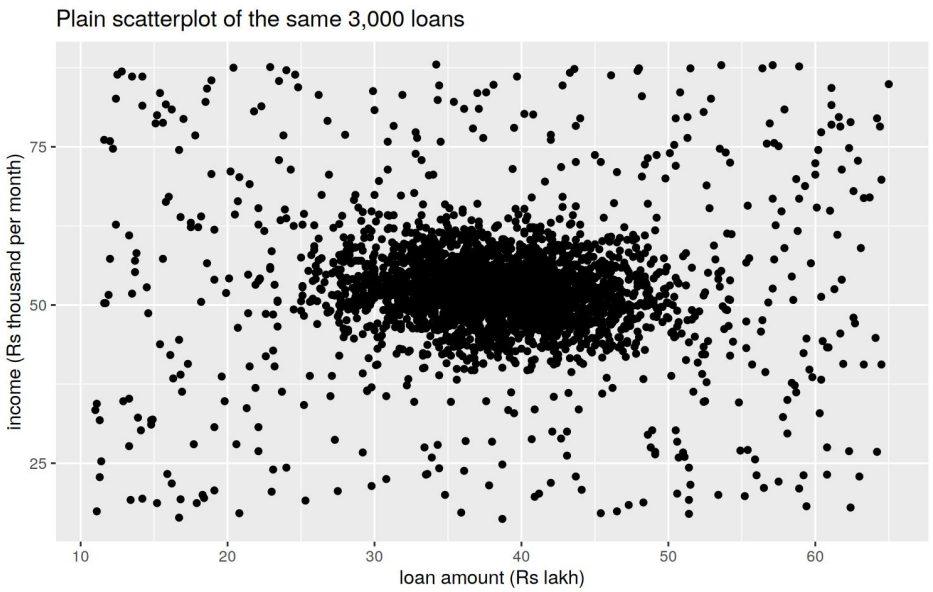
```
  loan_lakh income_k
1      39.7     86.1
2      40.9     54.9
3      37.3     56.0
4      46.7     58.6
5      23.1     48.5
6      38.0     49.6
7      57.5     75.1
8      31.7     49.4
[1] 3000
```

Task 1 [1.5]: Draw a hexbin plot (geom_hex(bins = 25)) with a count legend and a viridis scale; also draw the plain scatterplot of the same data.

```
ggplot(df, aes(loan_lakh, income_k)) +
  geom_hex(bins = 25) +
  scale_fill_viridis_c(name = "Count") +
  labs(title = "Hexbin plot: loan amount vs applicant income",
       x = "loan amount (Rs lakh)", y = "income (Rs thousand per month)")
ggplot(df, aes(loan_lakh, income_k)) +
  geom_point() +
  labs(title = "Plain scatterplot of the same 3,000 loans",
       x = "loan amount (Rs lakh)", y = "income (Rs thousand per month)")
```

OUTPUT:





Output 8: Hexbin plot (above) and plain scatterplot (below) of the same 3,000 loans

Ans: In the hexbin plot the colour shows how many loans fall in each hexagon: a bright, dense core near Rs 38 lakh and Rs 45 to 52 thousand, fading outwards. The plain scatterplot shows one solid blob in the middle, so the density inside it cannot be seen.

Task 2 [1.5]: Using a 10 x 10 grid (table(cut(x,10), cut(y,10))), find the busiest cell: give its x range, y range, count and share of all points.

```
tab <- table(cut(df$loan_lakh, 10), cut(df$income_k, 10))
w <- which(tab == max(tab), arr.ind = TRUE)
rownames(tab)[w[1, 1]]                          # loan range of busiest cell
colnames(tab)[w[1, 2]]                          # income range of busiest cell
max(tab)                                        # count in the cell
round(100 * max(tab) / nrow(df), 1)             # share of all points (%)
```

OUTPUT:

```
[1] "(38,43.4]"
[1] "(44.9,52.1]"
[1] 427
[1] 14.2
```

Ans: The busiest cell has loan amount (38, 43.4] lakh and income (44.9, 52.1] thousand. It holds 427 loans, which is 14.2% of all 3,000 points, in just one of the 100 grid cells.

Task 3 [1]: Report how many distinct (x, y) pairs there are (nrow(unique(df))) and explain why the scatterplot alone would mislead.

```
nrow(unique(df))                                # distinct (x, y) pairs
```

OUTPUT:

```
[1] 2906
```

Ans: There are 2,906 distinct (x, y) pairs out of 3,000 rows. Almost every point is distinct, but the points overlap on the page, so in the dense centre they merge into a solid area and the scatterplot cannot show that one cell holds 14% of all loans while the edges are sparse.

Task 4 [1]: Give one recommendation for the Zonal Manager.

Ans: Recommendation: build standard loan products and fast approval for the dense core (loans of about Rs 38 to 43 lakh, income Rs 45 to 52 thousand), where most applicants are, and handle the scattered high-loan and low-income cases individually with extra checks. Use hexbin or density plots, not plain scatterplots, for large loan data.